

DEVELOPMENT_RULES.md — Architecture, Workflow, Testing, Deployment

Scope: how to build. What to build lives in FUTURE_IDEAS.md; behavior specs in GAME_DESIGN.md and EDUCATION.md.

Architecture decision (authoritative)

- **Working base (keep):** the single-file `index.html` app is the current visual and functional MVP. Yulia likes the current design. **Do not delete or redesign it.** Preserve its look, features, content, and behavior.
- **Long-term canonical architecture: React + TypeScript.**
- **Migration rule:** gradual, feature-by-feature, only when needed, with **zero visual or behavioral regression**. The current app is the MVP and design reference — not disposable legacy code. The app will grow beyond math (multi-subject, per-child personalization, progress tracking, parent controls, adaptive difficulty), which is why the React/TS target exists.

Current code map (repo `summer-math-battle-tutor` , branch `main`)

Path	What it is
<code>index.html</code> (~4,700 lines)	The entire app: CSS in a <code><style></code> block, all markup, and one inline <code><script></code> with all logic. Sections are marked with <code>/* ===== NAME ===== */</code> banners — navigate by banner name, not line number.
<code>src/App.jsx</code> , <code>src/main.jsx</code> , <code>src/components/Particles.jsx</code> , <code>src/styles.css</code>	A non-mounting legacy React prototype — <code>index.html</code> has no <code>#root</code> and never loads <code>src/main.jsx</code> . It contains an <i>older</i> reward catalog and economy. Do not use it as a behavior reference; treat it only as scaffold space for the future React/TS migration.
<code>public/img/alex-hero.jpg</code> ,	Hero portraits, referenced by absolute path <code>/img/...</code>

<code>katya-hero.jpg</code>	(resolve on Vercel/preview, not <code>file://</code>).
<code>vite.config.js</code> , <code>vercel.json</code> , <code>package.json</code>	Vite dev/build (<code>npm run dev/build/preview</code>); React 18 + Playwright in deps.
<code>.github/workflows/npm-publish-github-packages.yml</code>	Boilerplate npm-publish workflow — not part of the app's deploy path.

Key script banners inside `index.html` (search for these): `GAME STATE`, `SOUND`, `ONTARIO CURRICULUM QUESTION ENGINE`, `ADAPTIVE DAILY MISSION ENGINE`, `BRAIN BOOST STRATEGY LIBRARY`, `STRATEGY LIBRARY SCREEN`, `MONEY LAB`, `GRANT PRIZE CHEST`, `GUIDED MISSION ENGINE`, `EQUATION ESCAPE ROOM ENGINE`, `BATTLE ARENA`, `NAV`, `WEEKEND DOUBLE XP`, `PROFILE PICKER`, `PARENT DASHBOARD` (render* functions).

State management (current)

- Single localStorage key: `smbt-state-v2`, via the `Store` wrapper (with in-memory fallback). Contains: both children's stats (`lvl`, `xp`, `xpMax`, `coins`, `streak`, `totalXP`, `keys`, `lastKeyMilestone`, per-topic mastery, recent-question buffer), active profile, editable reward catalog, all reward requests, activity log, parent PIN, settings (mute).
- **New fields must be back-filled** for existing saves (see the existing back-fill loop after `FRESH`) — never break an old save.
- `APP_VERSION` constant (currently `1.1.0`) + `__BUILD_ID__` placeholder power the in-app version/update check. Bump `APP_VERSION` on meaningful releases.
- localStorage is per-device/per-browser and is NOT the long-term architecture — see Backend path below.

Coding standards & conventions

- Match the existing style of `index.html`: small plain functions, `$` for `getElementById`, section banners, per-mode state objects (`M` mission, `B` battle, `GM` guided). New inline features follow this pattern.
- New React/TS code (migration work): functional components + hooks, TypeScript strict, one feature per folder, props typed, no `any`. Keep the neon-arcade design tokens identical to `index.html`.
- Naming: kid-facing strings are themed and warm; internal names are plain English (`topicWeight`, `renderStore`). Child ids are lowercase `alex` / `katya` everywhere.

- Never hardcode new personal/family details as string literals — route them through profile data so they can become parent-editable fields (CHARACTERS.md → Privacy rules).
- Known debt to fix, not copy: `markMissionDone` levels with `xpMax × 1.15` while everywhere else uses `+50` (see FUTURE_IDEAS.md → Now).

Git workflow

- **Feature branches + pull requests for anything meaningful.** Anything affecting logic, data, profiles, rewards, learning flows, navigation, or design requires: (1) clearly named feature branch, (2) focused commits, (3) a PR, (4) review before merge, (5) merge only after the app works.
- Only typo fixes, docs-only changes, and very minor safe updates may go directly to `main`.
- `main` must always remain stable and usable by the children.
- AI developers (Claude Code, Codex, etc.) may create branches, commit, and prepare PRs — but must **not merge major work without Yulia's review**.

Testing (Playwright — practical, not bureaucratic)

- No giant suite for tiny changes; every meaningful feature or bug fix adds or updates tests for the affected **child-critical flow**.
- Minimum coverage to build toward (baseline suite is a Now item in FUTURE_IDEAS.md): app loads; profile selection works; Daily Mission starts; questions display; answers enterable on tablet and mobile; correct answers progress; wrong answers follow the 3-attempt teaching flow (EDUCATION.md); hints work; reinforcement questions appear; XP/coins/streaks update; progress saves; rewards viewable and redeemable correctly; parent controls stay protected; navigation never traps the child; refresh doesn't destroy progress.

Pre-merge checklist for meaningful PRs

1. Run existing automated tests.
2. Add/update tests for changed critical behavior.
3. Manually test on at least one tablet-sized screen.
4. Confirm no visual design regression.

5. Confirm existing child data and progress are preserved.

6. Confirm the production build succeeds.

Visual-only changes (lighter rule): check the affected screen at tablet and mobile sizes; no layout regression; text readable and controls touch-friendly.

Devices & responsive requirements

Alex: iPad. Katya: Lenovo Android tablet. Parent: iPhone + desktop. **Tablet-first**, must also work on mobile and desktop. Every important screen: touch usability, large-enough buttons, readable text, no horizontal scrolling, no cut-off content, no broken keyboard input, no hover-only interactions, good performance on older devices, safe orientation changes.

Deployment

- Auto-deploy from `main` via **Vercel** (framework preset Vite, build `npm run build`, output `dist`).
- **Live URL:**  **TO BE RECORDED — confirm from the Vercel dashboard or repo settings and write it here. Do not guess.**
- Change the parent PIN from the default `1234` on the production deployment.

Backend path (approved direction — build when features demand it)

Keep local-only MVP working, but `localStorage` is not the destination. When worksheet uploads, AI tutoring, assignment planning, cross-device progress, parent analytics, or personalized recommendations arrive, move to:

- Family account with separate child profiles
- Secure cloud database; progress synced across devices
- File storage for uploaded homework/worksheets/teacher materials
- **AI API called only through the backend — never exposed in the client**
- Parent-controlled access to all child data; export and delete options
- Local fallback / offline support where practical (session continues offline, syncs later)

Migrate gradually — replace `localStorage` piece by piece; preserve design and working features.

Before replacing localStorage: write a safe migration plan for existing progress and profiles; never erase current data; back up before major storage changes; test sync and account recovery; make failures understandable to the parent.

Provider guidance (nothing locked in yet): reasonable starting direction is Vercel (deploy) + Supabase or Firebase (auth, DB, file storage) + a server-side AI API (OpenAI or Claude). Before choosing, compare privacy, cost, ease of development, Canadian availability, file handling, and long-term portability. **Create a provider abstraction for the AI model** — never hard-code the app around one provider.

Data & privacy rules (children's data — mandatory)

- Children's data stays private to the family account. Never use it for public examples, training datasets, demos, or third-party analytics.
- No API keys in client-side code.
- Send an AI provider no more personal information than the immediate task requires; strip unnecessary names/identifying details from AI requests.
- Uploaded school documents are private by default.
- Parents can view, export, and permanently delete stored data.
- Parent controls and uploaded documents are access-protected.
- Real names, birthdays, school materials, and progress never appear in public repositories or logs.
- Encryption in transit; secure provider-managed storage.
- Keep a clear record of what data is stored and which services process it.
- No advertising, tracking pixels, or unnecessary third-party analytics.

Cost rules

- Free tiers where reliable; target **under CAD \$20/month** during normal family use. **Ask Yulia before adding any service expected to exceed that.**
- Add usage limits, file-size limits, and AI request controls so costs can't spike. Parent dashboard should eventually show AI usage and estimated cost.
- Don't reduce educational quality merely to avoid all cost — but no unnecessary enterprise services.

